

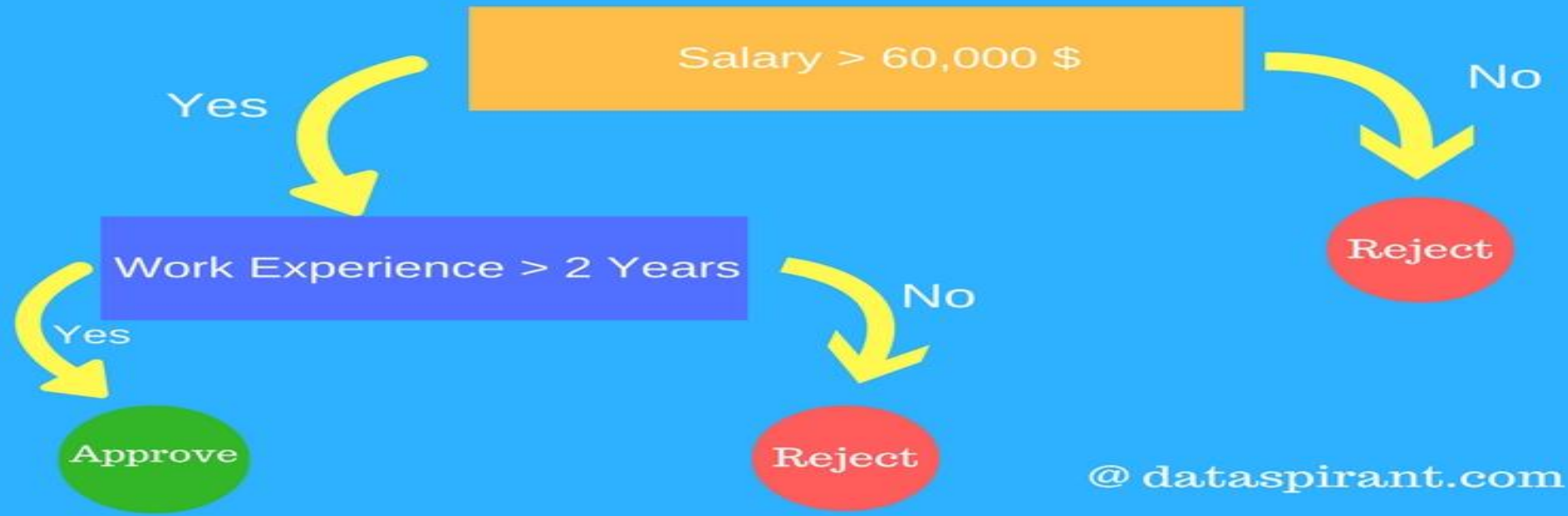
Decision Tree

Decision



Algorithm

Should We Issues Loan?

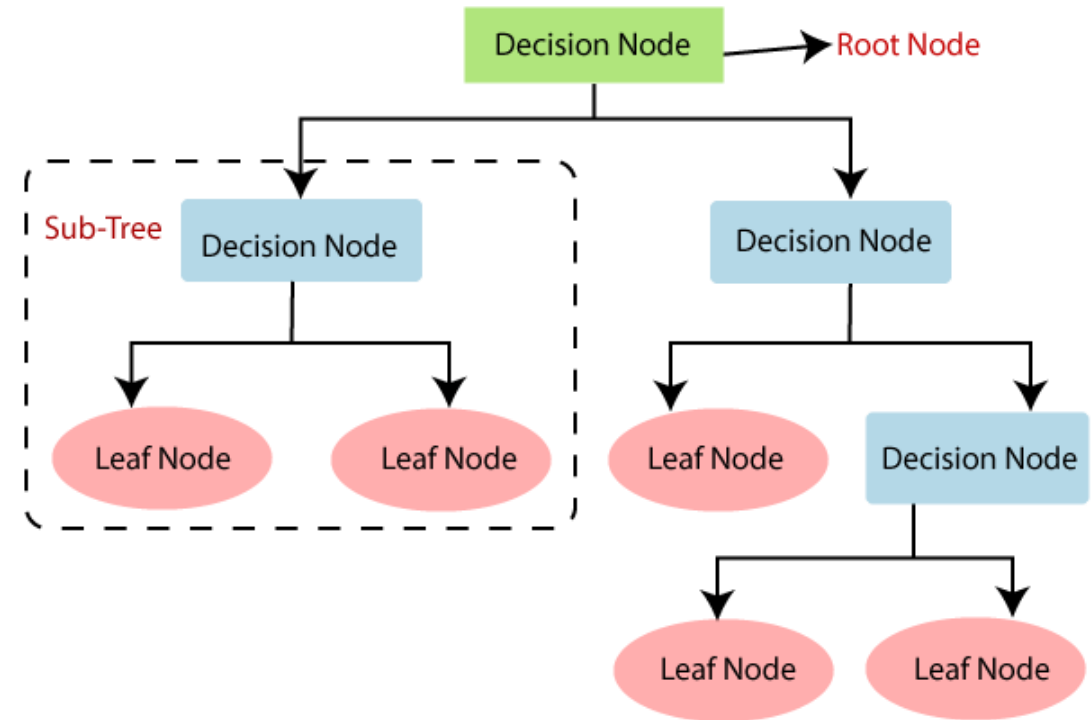


Decision Tree algorithm

- Decision Tree algorithm belongs to the family of supervised learning algorithms.
- Unlike other supervised learning algorithms, decision tree algorithm can be used for solving regression and classification problems too.

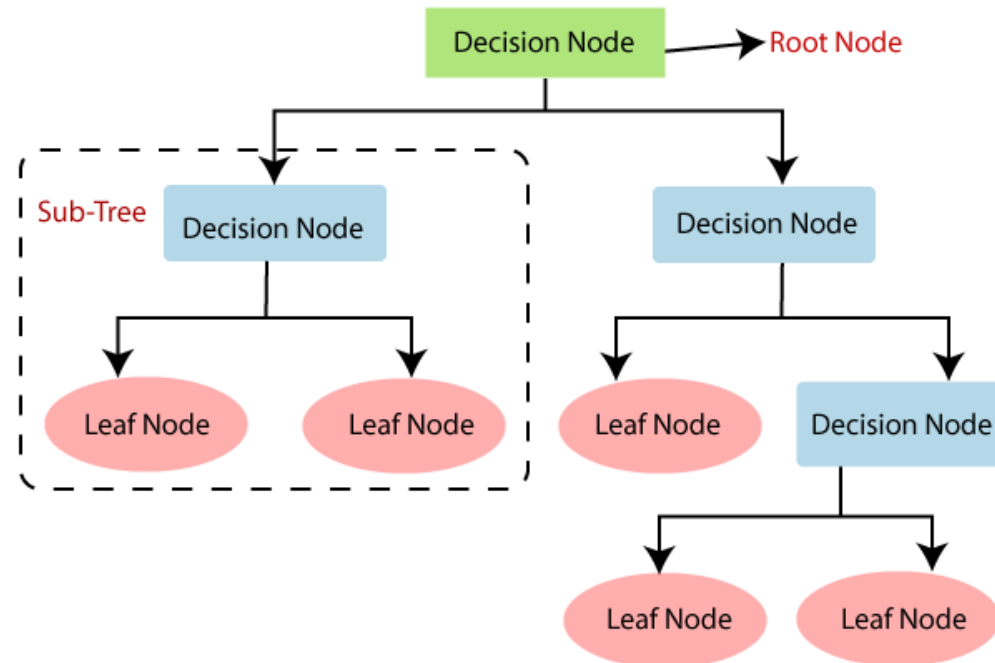
Decision Tree

- **A root node:** this is the node that begins the splitting process by finding the variable that best splits the target variable
- **Node purity:** Decision nodes are typically impure, or a mixture of both classes of the target variable (0,1 or green and red dots in the image). Pure nodes are those that have one class — hence the term pure. They either have green or red dots only in the image.
- **Decision nodes:** these are subsequent or intermediate nodes, where the target variable is again split further by other variables
- **Leaf nodes** or **terminal nodes:** are pure nodes, hence are used for making a prediction of a numerical or class is made.



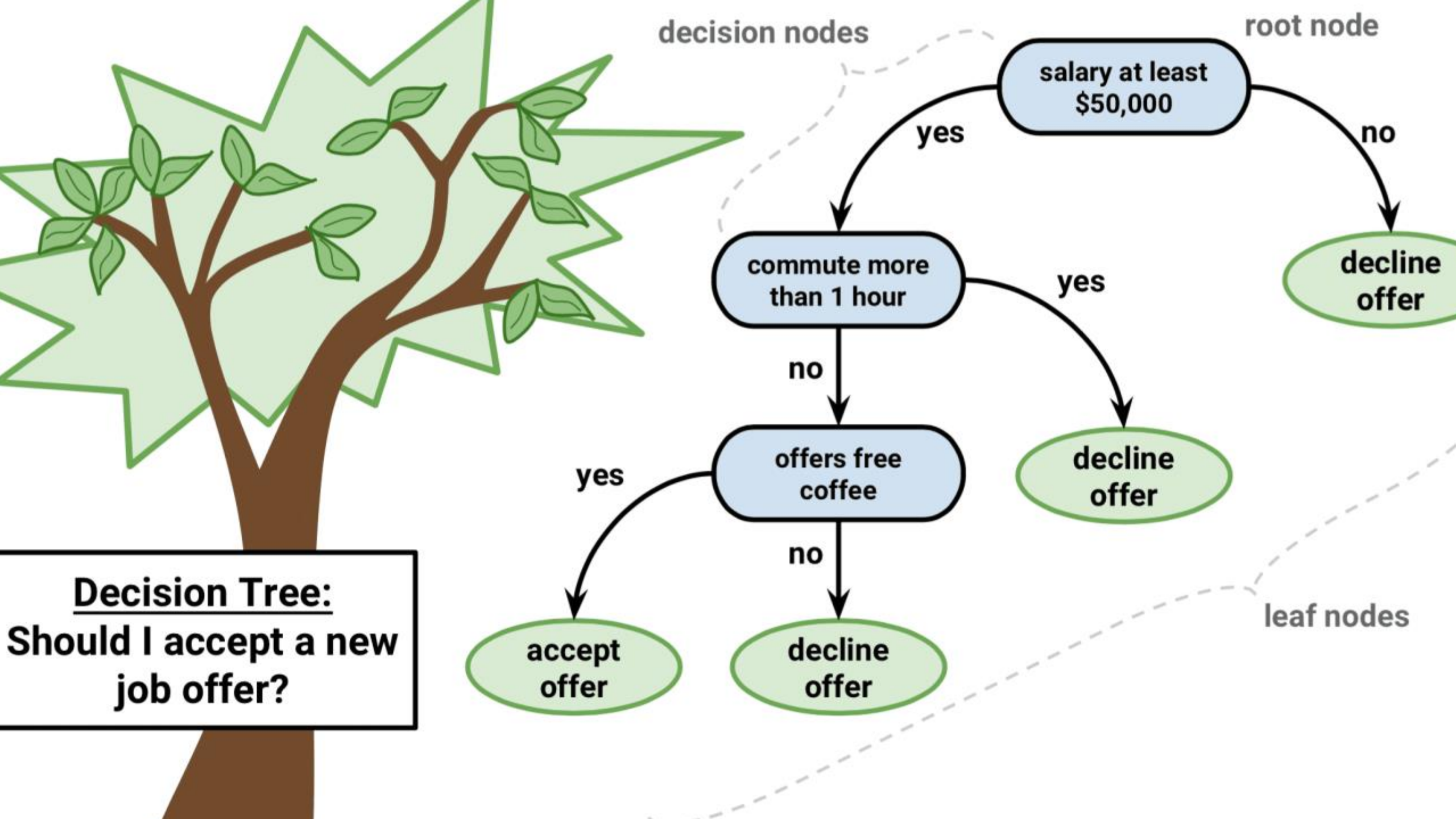
Decision Tree Algorithm Pseudocode

1. Place the best attribute of the dataset at the **root** of the tree.
2. Split the training set into **subsets**. Subsets should be made in such a way that each subset contains data with the same value for an attribute.
3. Repeat step 1 and step 2 on each subset until you find **leaf nodes** in all the branches of the tree.



Decision Tree Algorithm Pseudocode

- Assign all training instances to the root of the tree. Set current node to root node.
- For each attribute
 - **Partition** all data instances at the node by the value of the attribute.
 - **Compute** the **information gain ratio** from the partitioning.
- Identify feature that results in the **greatest information gain** ratio. Set this feature to be the **splitting criterion** at the current node.
 - If the best **information gain ratio** is 0, tag the current node as a **leaf** and return.
- Partition all instances according to attribute value of the best feature.
- Denote each partition as a child node of the current node.
- For each child node:
 - If the child node is “**pure**” (has instances from only one class) tag it as a **leaf** and return.
 - If not set the child node as the current node and recurse to step 2



Splitting criteria

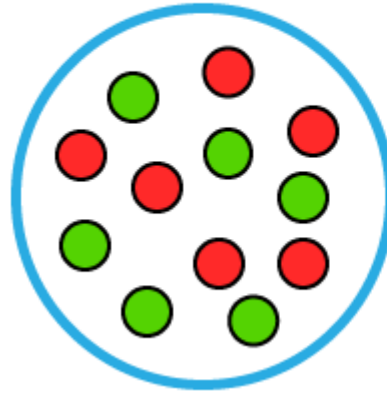
- In decision trees, There are some methods to determine the splitting criteria as We need something to **measure** and **compare** the **impurity**.
- Some of the methods are **Gini Impurity, Entropy, Information Gain, Gain Ratio, Reduction in variance, Chi square** .

Entropy

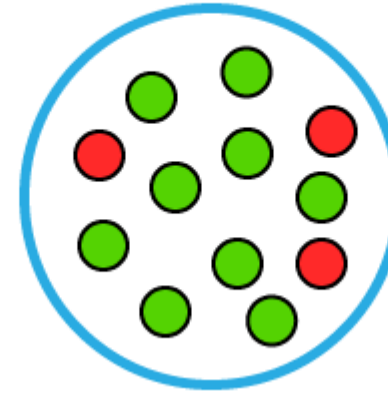
- **Entropy** is a scientific concept as well as a measurable physical property that is most commonly associated with a state of **disorder**, **randomness**, or **uncertainty**.
- In information theory, the **entropy** of a random variable is the average level of “**surprise**”, or “**uncertainty**” inherent to the variable’s possible outcomes.
- In the context of Decision Trees, entropy is a measure of disorder or **impurity** in a node.
- Thus, a node with more variable composition, such as 2Pass and 2 Fail would be considered to have higher Entropy than a node which has only pass or only fail.

Entropy

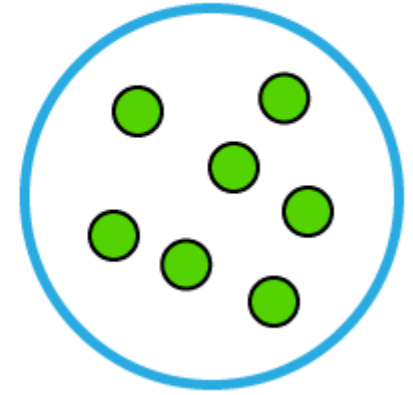
Very Impure



Less Impure



Minimum Impurity



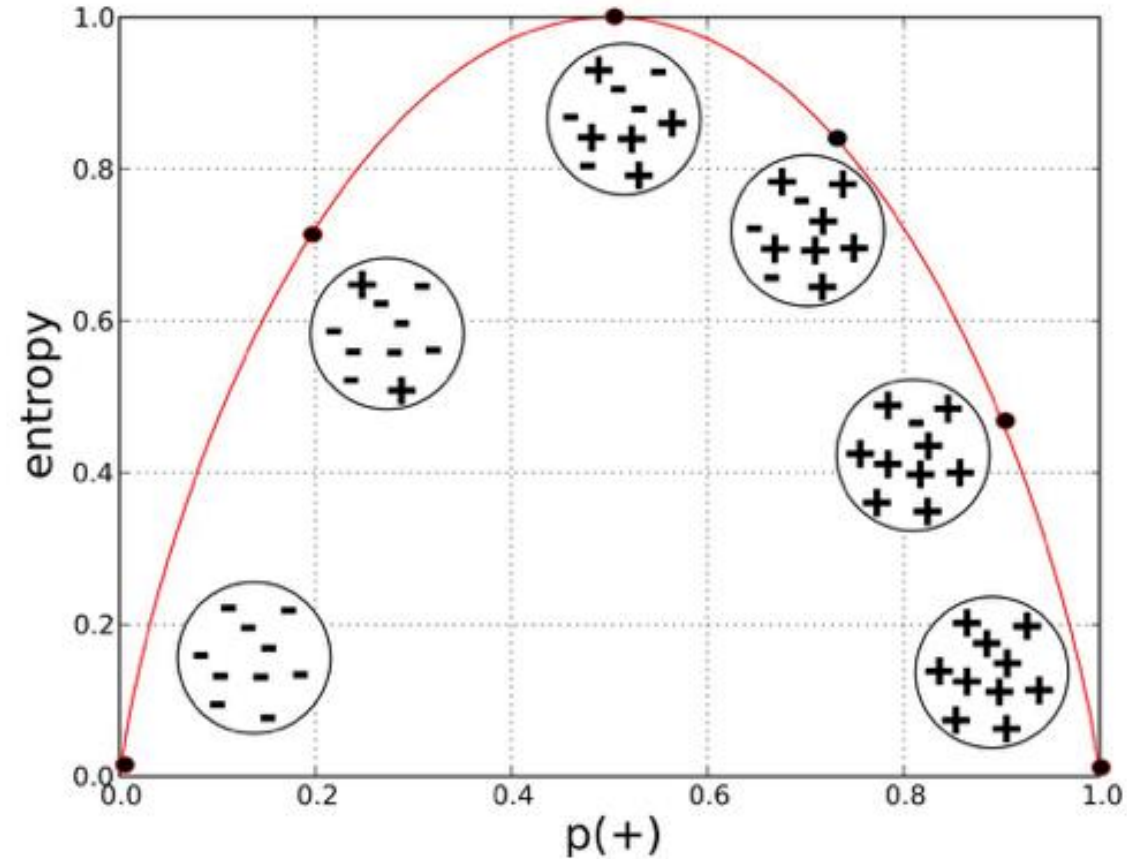
$$E = - \sum_{i=1}^n p_i \log_2(p_i)$$

P_{pass} = Probability of passing/Total no. of instances = 9/15

P_{fail} = Probability of failing/Total no. of instances = 6/15

$$E = - \left[P_{\text{pass}} \log_2(P_{\text{pass}}) + P_{\text{fail}} \log_2(P_{\text{fail}}) \right]$$

Entropy



- The **maximum** level of entropy or disorder is given by **1** and **minimum** entropy is given by a value **0**.
- **Leaf nodes** which have all instances **belonging** to **1 class** would have an entropy of **0**.
- Whereas, the entropy for a node where the classes are **divided equally** would be **1**.

Entire population (30 instances)



● : 16
★ : 14

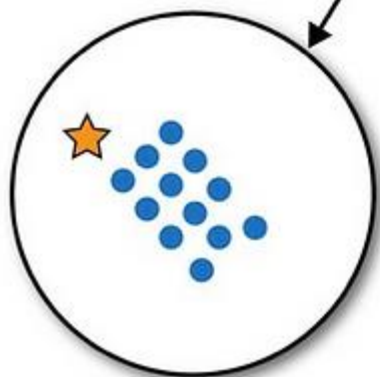
$p(\bullet) = 16/30 \approx 0.53$
 $p(\star) = 14/30 \approx 0.47$

$$E(\text{Parent}) = -\frac{16}{30}\log_2\left(\frac{16}{30}\right) - \frac{14}{30}\log_2\left(\frac{14}{30}\right) \approx 0.99$$

$$E(\text{Balance} < 50K) = -\frac{12}{13}\log_2\left(\frac{12}{13}\right) - \frac{1}{13}\log_2\left(\frac{1}{13}\right) \approx 0.39$$

$$E(\text{Balance} > 50K) = -\frac{4}{17}\log_2\left(\frac{4}{17}\right) - \frac{13}{17}\log_2\left(\frac{13}{17}\right) \approx 0.79$$

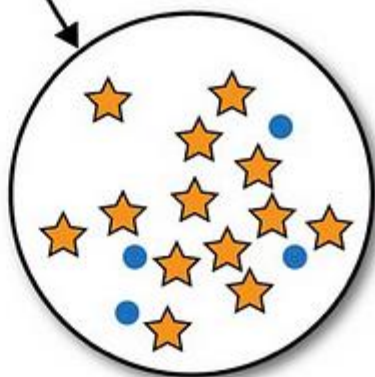
Balance < 50K



● : 12
★ : 1

$p(\bullet) = 12/13 \approx 0.92$
 $p(\star) = 1/13 \approx 0.08$

Balance ≥ 50K



● : 4
★ : 13

$p(\bullet) = 4/17 \approx 0.24$
 $p(\star) = 13/17 \approx 0.76$

Information Gain

Which feature should we test to add a new node?

- Information is anything that **decreases** our **uncertainty**.
- The gain cannot be **negative**
- The feature with the **largest** expected amount is the **best** choice.

$$\text{Information Gain} = \text{Entropy}_{\text{parent}} - \text{Entropy}_{\text{children}}$$

$$IG(Y, X) = E(Y) - E(Y|X)$$

Entire population (30 instances)



● : 16
★ : 14

$p(\bullet) = 16/30 \approx 0.53$
 $p(\star) = 14/30 \approx 0.47$

$$E(\text{Parent}) = -\frac{16}{30}\log_2\left(\frac{16}{30}\right) - \frac{14}{30}\log_2\left(\frac{14}{30}\right) \approx 0.99$$

$$E(\text{Balance} < 50K) = -\frac{12}{13}\log_2\left(\frac{12}{13}\right) - \frac{1}{13}\log_2\left(\frac{1}{13}\right) \approx 0.39$$

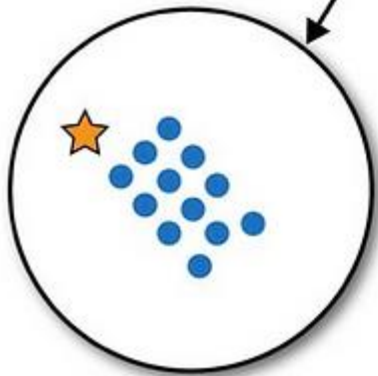
$$E(\text{Balance} > 50K) = -\frac{4}{17}\log_2\left(\frac{4}{17}\right) - \frac{13}{17}\log_2\left(\frac{13}{17}\right) \approx 0.79$$

Weighted Average of entropy for each node:

$$\begin{aligned} E(\text{Balance}) &= \frac{13}{30} \times 0.39 + \frac{17}{30} \times 0.79 \\ &= 0.62 \end{aligned}$$

$$\begin{aligned} IG(\text{Parent}, \text{Balance}) &= E(\text{Parent}) - E(\text{Balance}) \\ &= 0.99 - 0.62 \\ &= 0.37 \end{aligned}$$

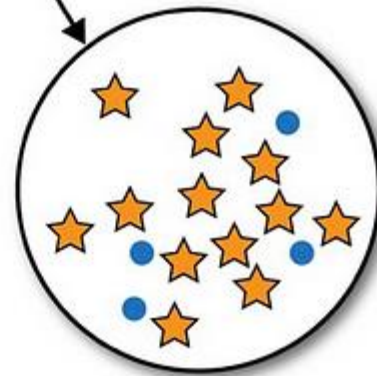
Balance < 50K



● : 12
★ : 1

$p(\bullet) = 12/13 \approx 0.92$
 $p(\star) = 1/13 \approx 0.08$

Balance ≥ 50K



● : 4
★ : 13

$p(\bullet) = 4/17 \approx 0.24$
 $p(\star) = 13/17 \approx 0.76$

Information Gain

$$IG(t) = -\sum_{i=1}^m p(c_i) \log p(c_i) \\ + p(t) \sum_{i=1}^m p(c_i | t) \log p(c_i | t) + p(\bar{t}) \sum_{i=1}^m p(c_i | \bar{t}) \log p(c_i | \bar{t})$$

c_i represents the i th category, $P(c_i)$ is the probability of the i th category.

$P(t)$ and $P(\bar{t})$ are the probabilities that the term t appears or not in the documents.

$P(c_i|t)$ is the conditional probability of the i th category given that term t appeared, and $P(c_i|\bar{t})$ is the conditional probability of the i th category given that term t does not appeared.

Age	Mileage	Road Tested	Buy
Recent	Low	Yes	Buy
Recent	High	Yes	Buy
Old	Low	No	Don't buy
Recent	High	No	Don't buy

Age	Mileage	Road Tested	Buy
Recent	Low	Yes	Buy ✓
Recent	High	Yes	Buy ✓
Old	Low	No	Don't buy ✗
Recent	High	No	Don't buy ✗

Root node

✓ 2 instances
✗ 2 instances

Calculating Entropy for the root node

$$E = - \left(P(\checkmark) * \log_2(P(\checkmark)) + P(\times) * \log_2(P(\times)) \right)$$

Probability formula:

$$P(\checkmark) = \frac{\text{count of } \checkmark}{\text{total examples}}$$

$$P(\checkmark) = 2/4 = 0.5$$

$$P(\times) = 2/4 = 0.5$$

Plugging these values in the formula we get:

$$E = - \left(0.5 * \log_2(0.5) + 0.5 * \log_2(0.5) \right)$$

$$E = 1$$

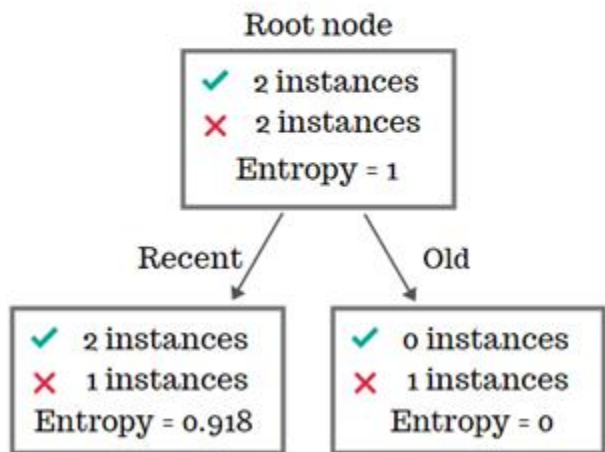
Information Gain (IG)

$$IG = \text{Entropy (Parent)} - \text{weighted_avg} * \text{Entropy (Children)}$$

$$\text{Entropy (Parent)} = 1$$

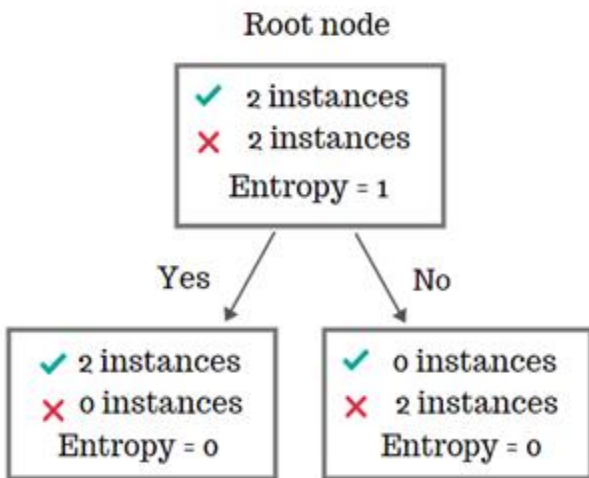
Information gain for Age

Age	Mileage	Road Tested	Buy
Recent	Low	Yes	Buy ✓
Recent	High	Yes	Buy ✓
Old	Low	No	Don't buy ✗
Recent	High	No	Don't buy ✗



Information gain for Road Tested

Age	Mileage	Road Tested	Buy
Recent	Low	Yes	Buy ✓
Recent	High	Yes	Buy ✓
Old	Low	No	Don't buy ✗
Recent	High	No	Don't buy ✗

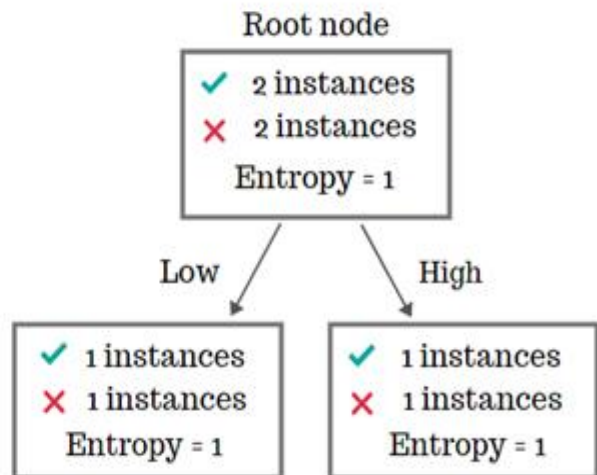


$$\text{Children Entropy} = \frac{2}{4}(0) + \frac{2}{4}(0) = 0$$

$$\text{Information gain} = 1 - 0 = 1$$

Information gain for Mileage

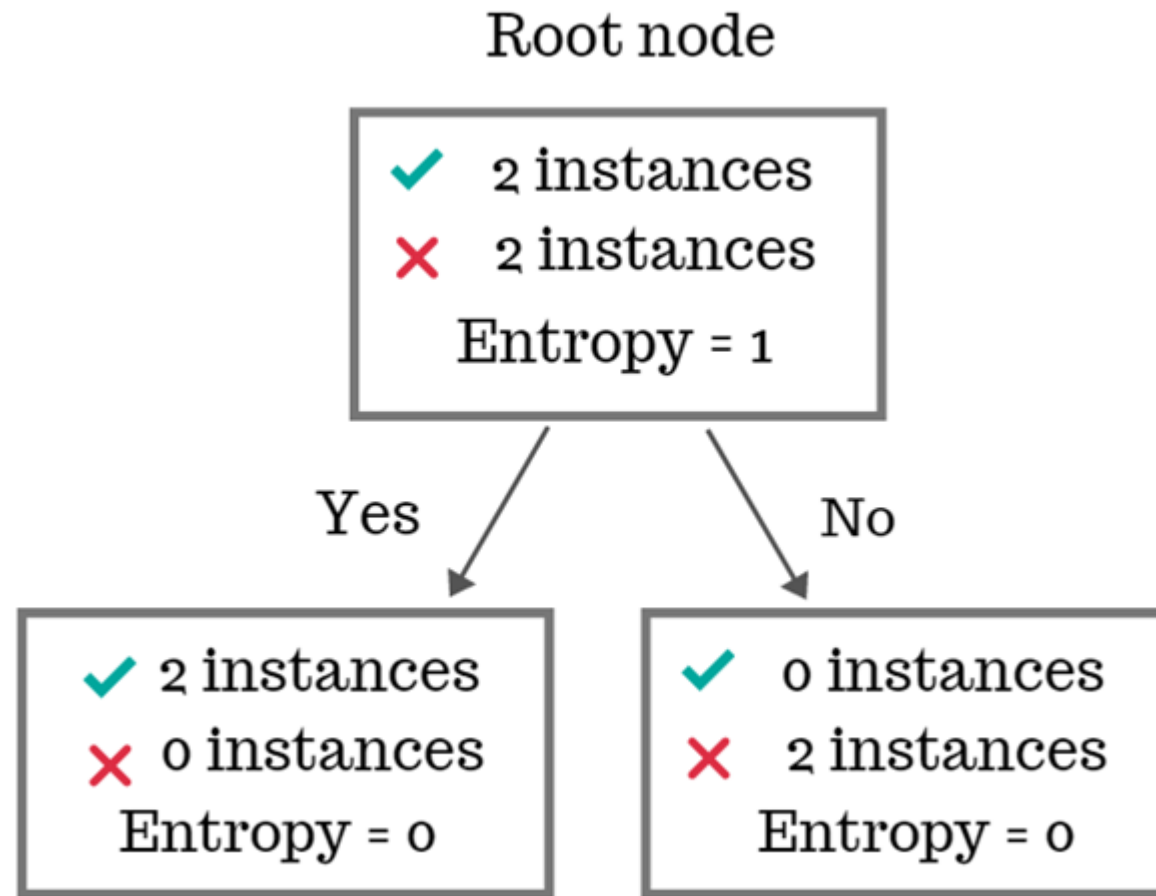
Age	Mileage	Road Tested	Buy
Recent	Low	Yes	Buy ✓
Recent	High	Yes	Buy ✓
Old	Low	No	Don't buy ✗
Recent	High	No	Don't buy ✗



$$\text{Children Entropy} = \frac{2}{4}(1) + \frac{2}{4}(1) = 1$$

$$\text{Information gain} = 1 - 1 = 0$$

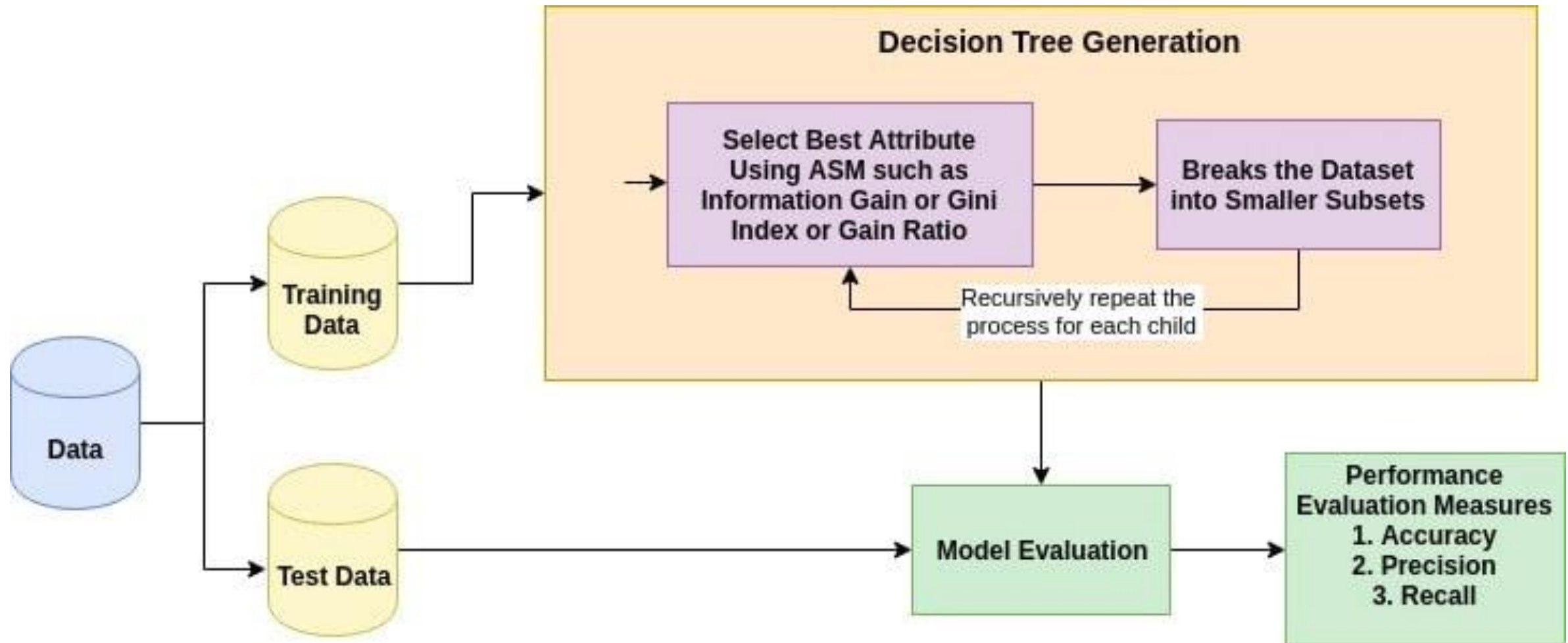
	Age	Mileage	Road Tested	Buy
	Recent	Low	Yes	Buy
	Recent	High	Yes	Buy
	Old	Low	No	Don't buy
	Recent	High	No	Don't buy
Info. Gain		0.3112	0	1



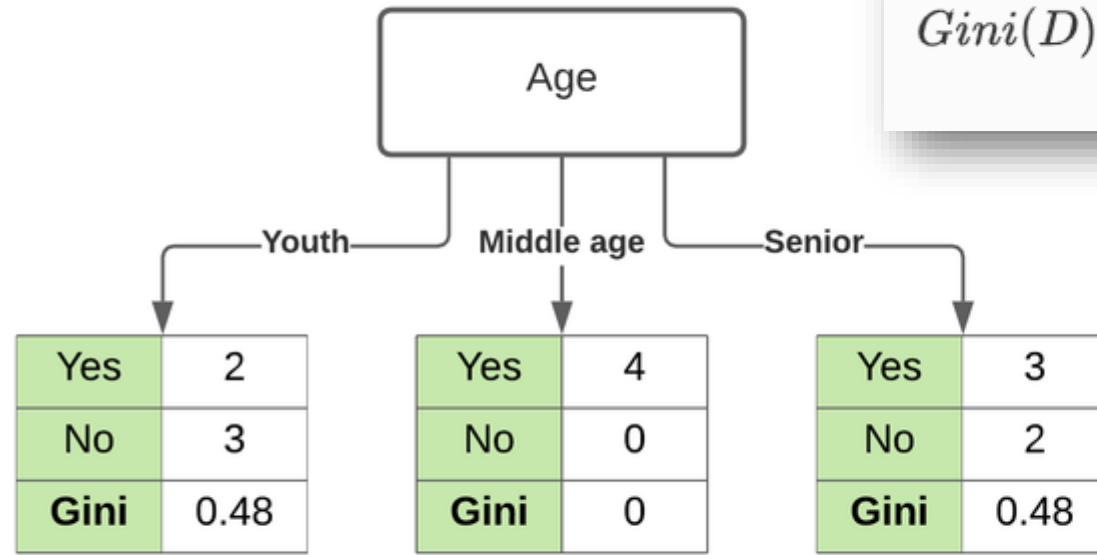
Since the entropy is **zero** at the leaf nodes, we'll **stop splitting**.

If this wasn't the case, then we would **continue** to find the **information gain** for the **preceding parent and children nodes**, until any one of the stopping criteria is met.

Decision Tree



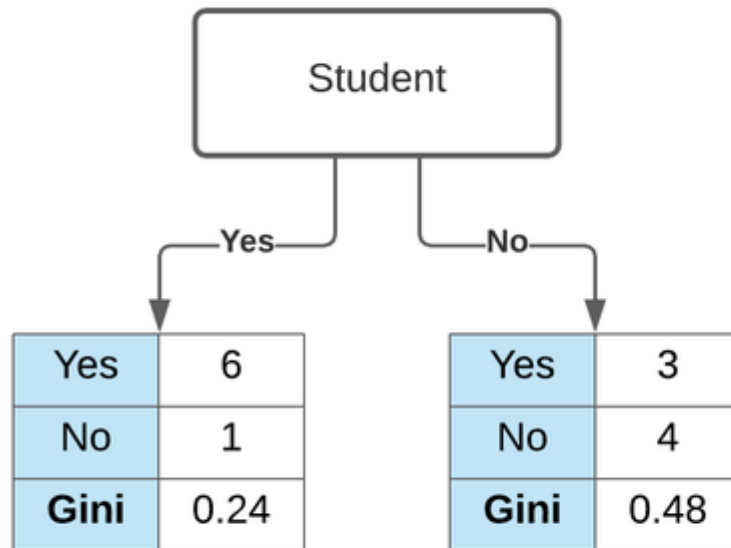
$$Gini(D) = 1 - \sum_{i=1}^k p_i^2$$



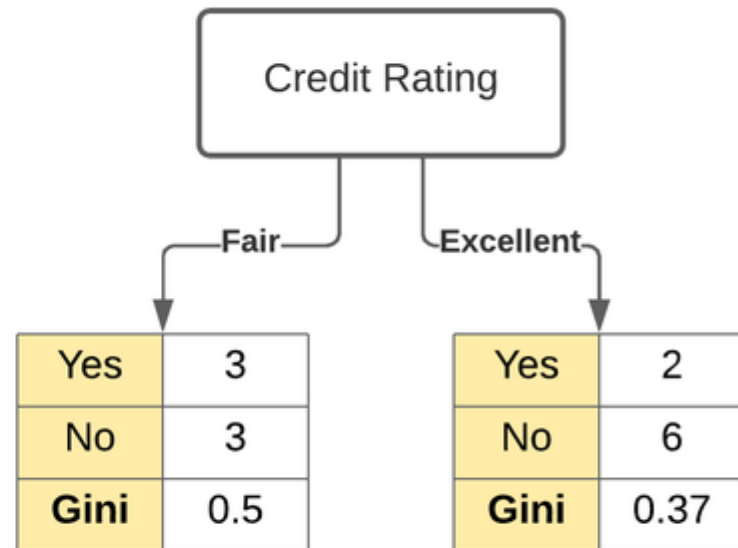
Gini Impurity for Age is 0.343



Gini Impurity for Income is 0.440



Gini Impurity for Student is 0.367



Gini Impurity for Credit Rating is 0.429

Best

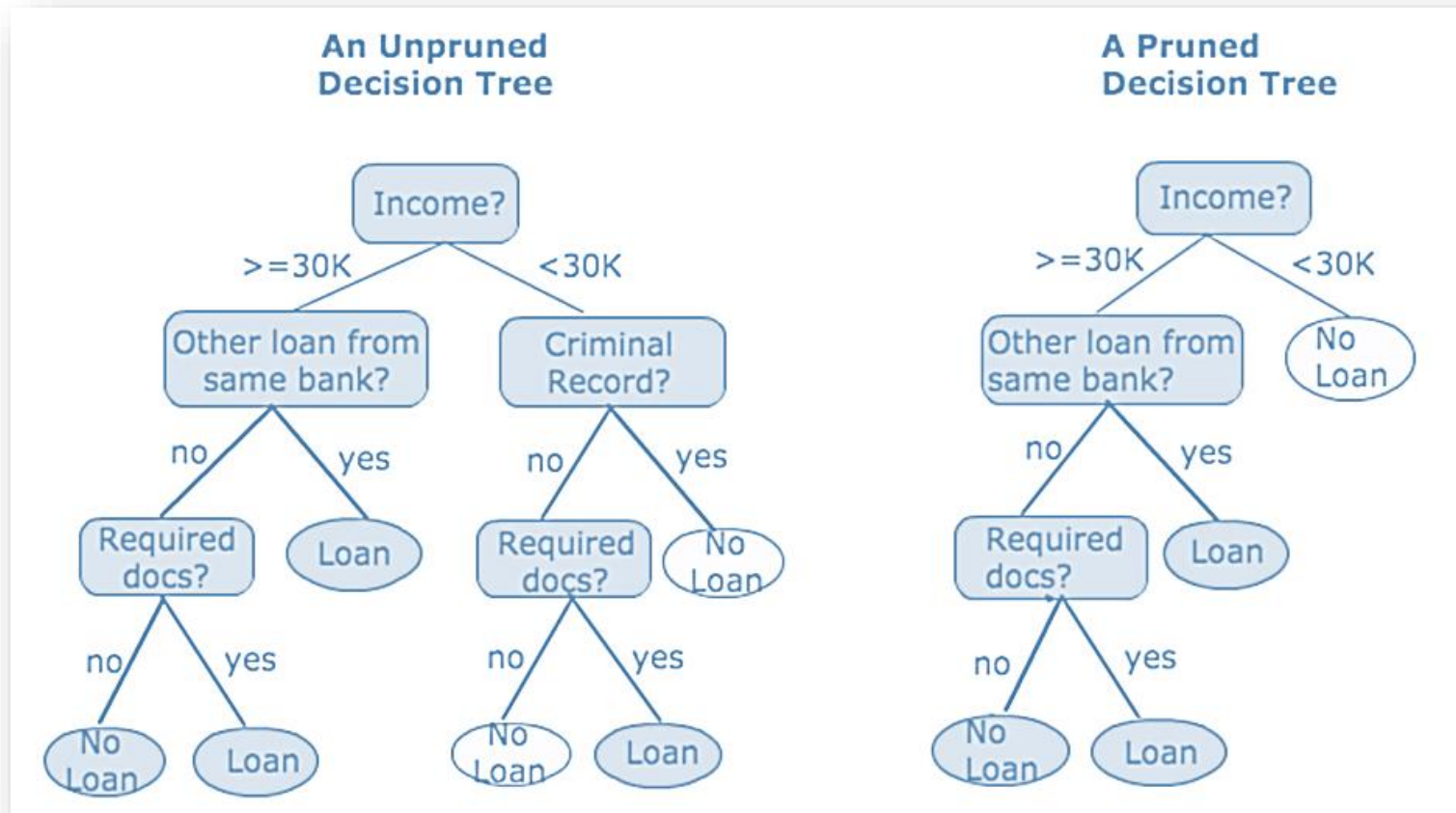
Advantages of Decision Tree

- Normalization or **Scaling** of data is **not required**
- Easy to **explain** and **visualize** (It results in a set of rules).
- Automatic **feature selection** i.e. irrelevant features won't affect decision trees.
- The Number of **hyper-parameters** to be tuned is almost **null**.
- Work well with **tabular data** (structured data)

Disadvantages of Decision Tree

- **Sensitive** to **data** i.e. If data changes slightly, outcome also change to a very large extend.
- Requires **more time** in training
- There is a **high probability** of **overfitting** in Decision Tree.
- Generally, it gives **low prediction accuracy** for a dataset as compared to other machine learning algorithms.
- Calculations can become **complex** when there are **many class labels**.

Pruning decision trees



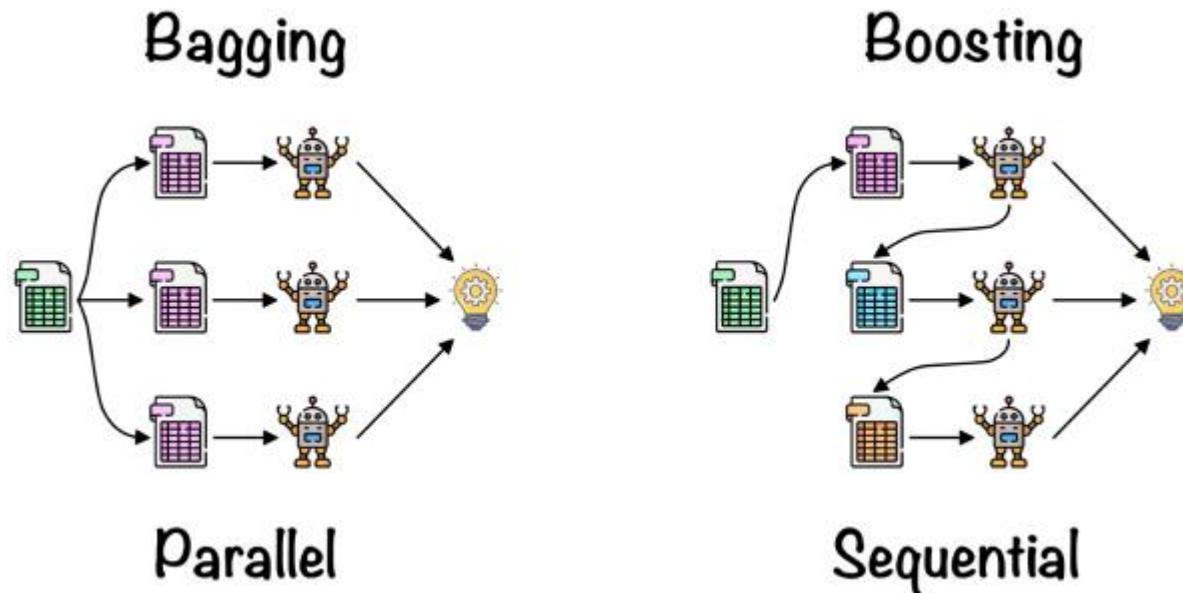
The process of **IG-based pruning** requires us to identify “**twigs**”, nodes whose children are all leaves. “Pruning” a twig removes all of the leaves which are the children of the twig, and makes the twig a leaf.

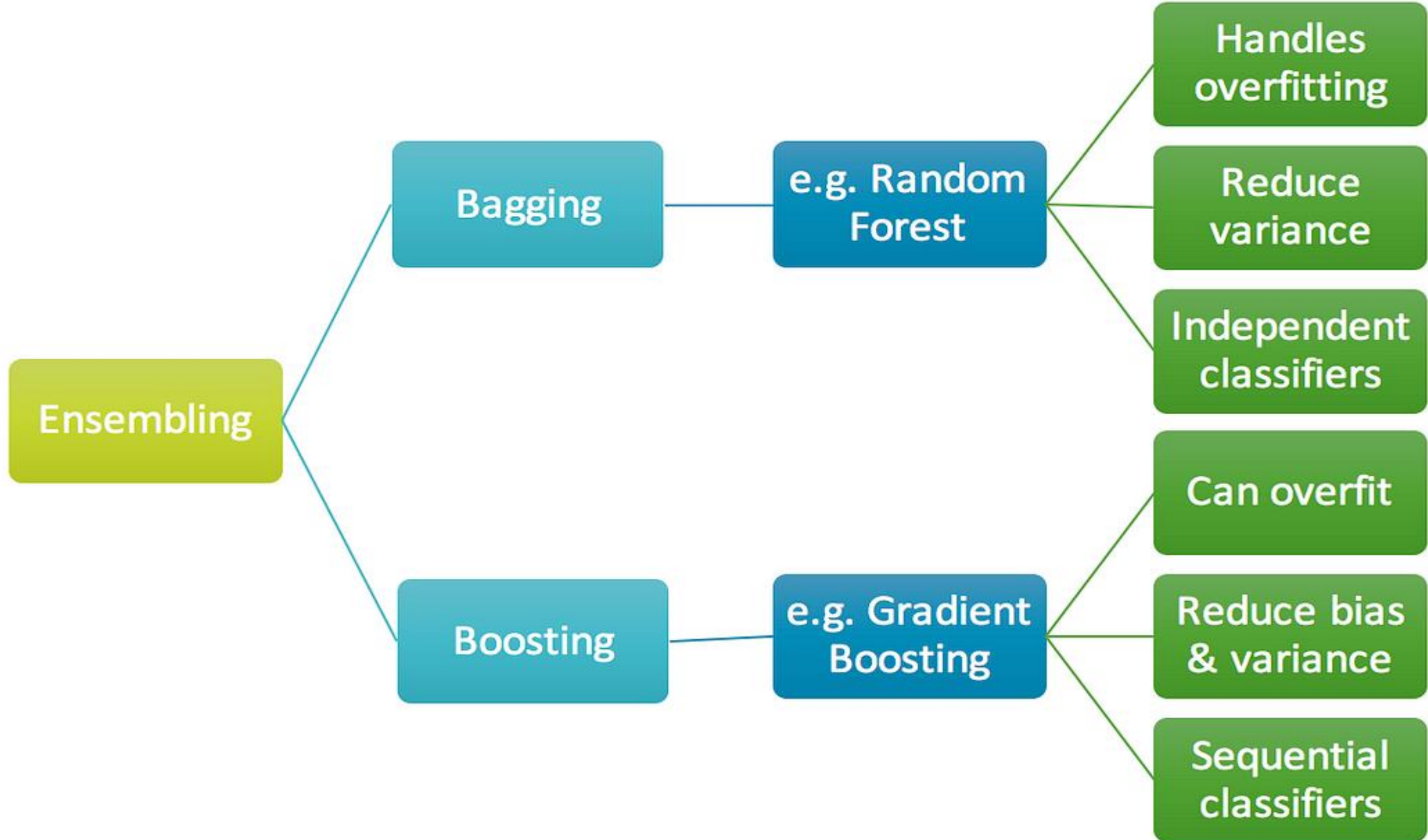
DecisionTreeClassifier in SkLearn

- The hyperparameters of the DecisionTreeClassifier in SkLearn include `max_depth`, `min_samples_leaf`, `min_samples_split` which can be tuned to early stop the growth of the tree and prevent the model from overfitting.
- The best way is to use the sklearn implementation of the `GridSearchCV` technique to find the best set of hyperparameters for a Decision Tree model.

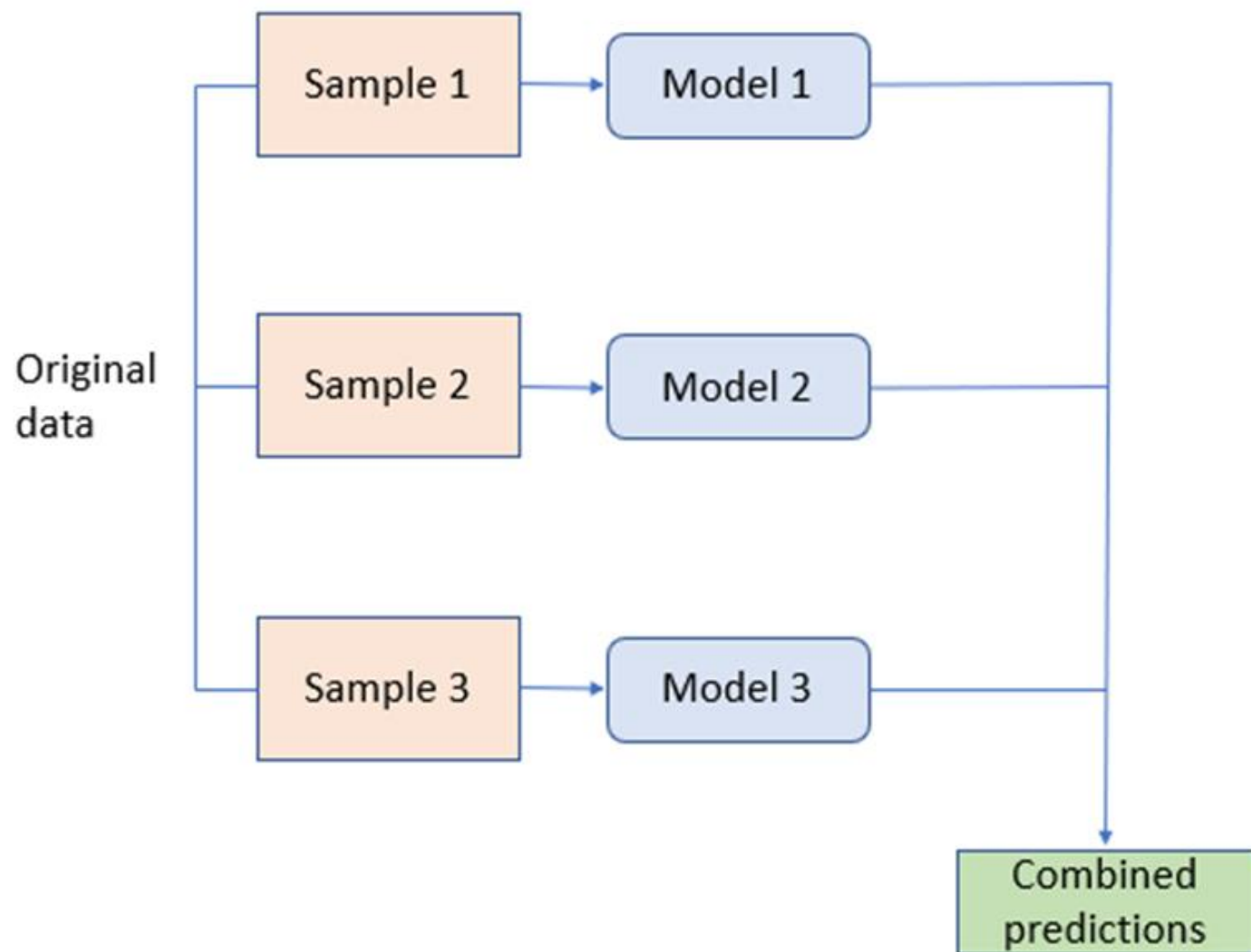
Ensemble learning

- **Ensemble learning** is a machine learning paradigm where **multiple models** (learners) are trained to solve the **same problem**.
- By using multiple learners, **generalization** ability of an ensemble can be much better than single learner.



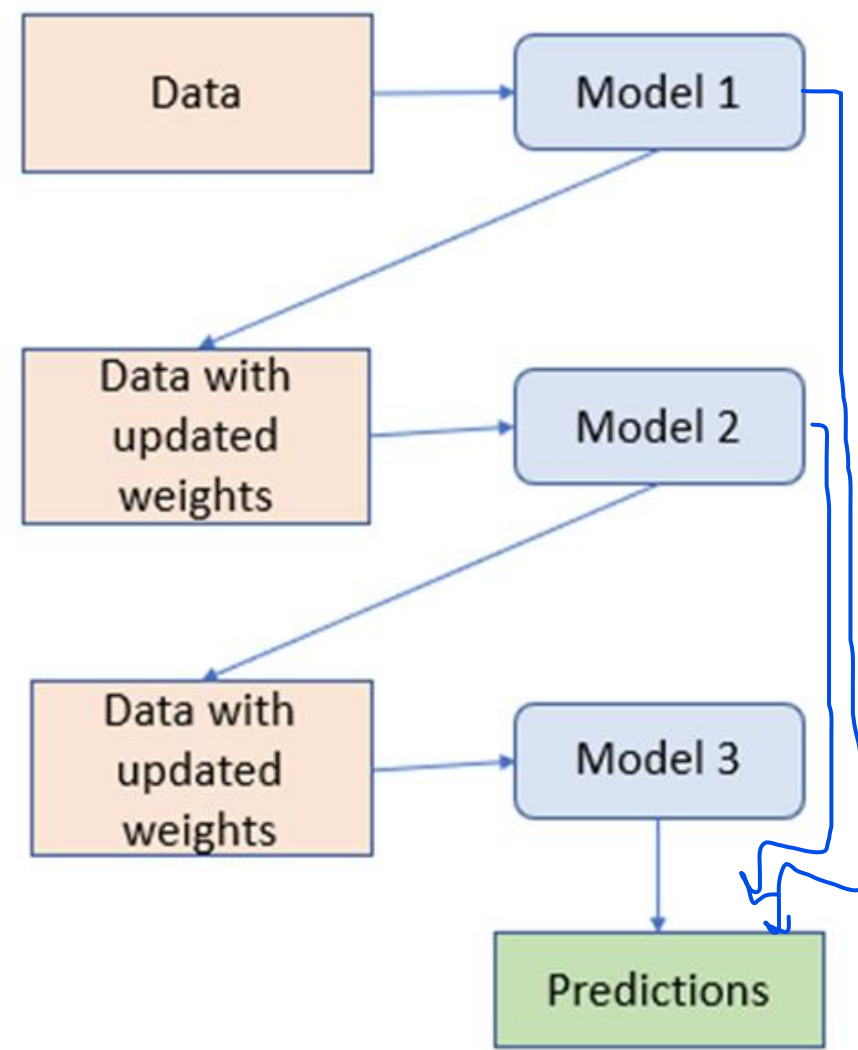


Bagging



★ Bagging is used mainly in reducing variance and it is a parallel process

Boosting

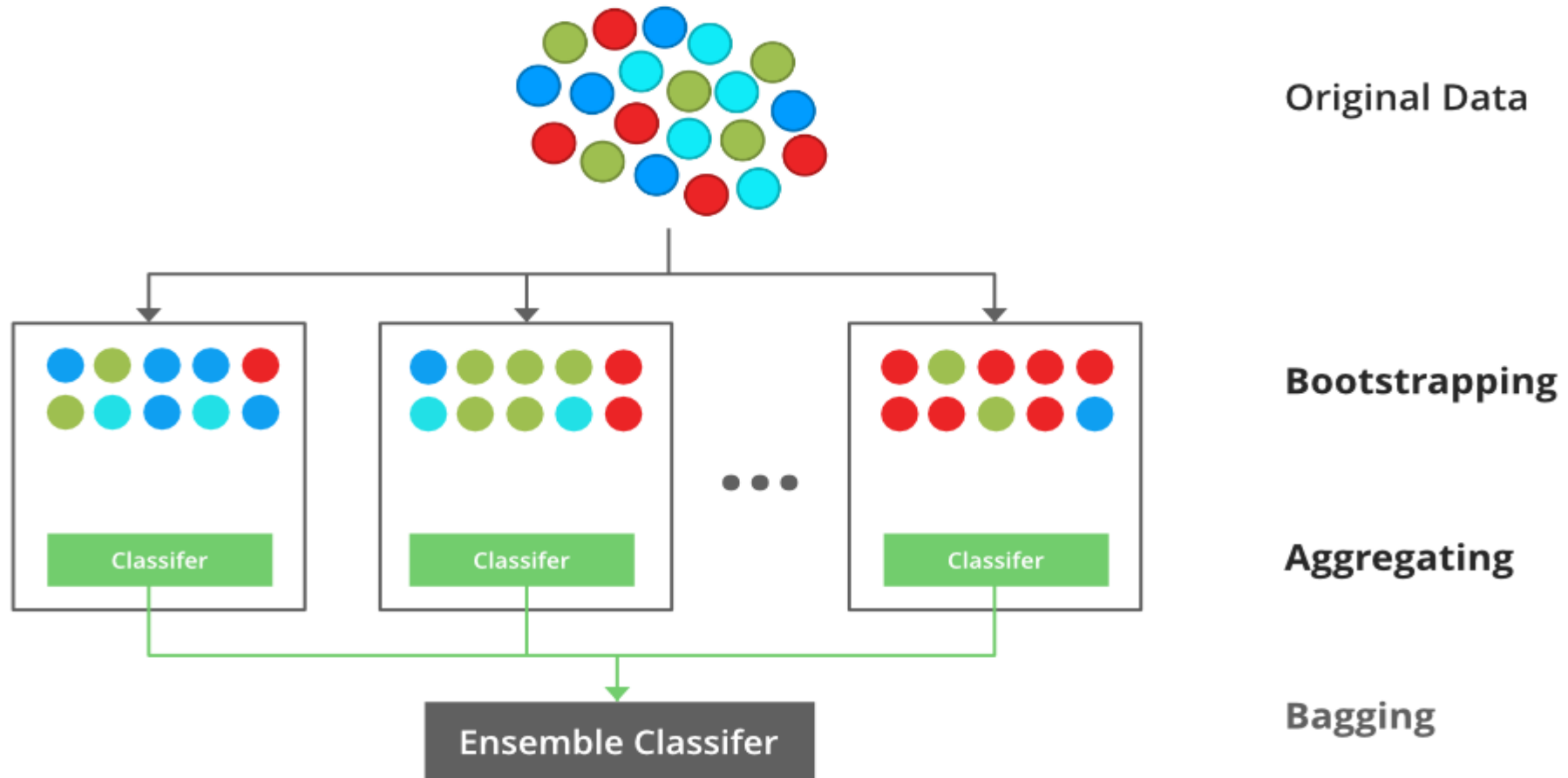


★ Boosting is used mainly in reducing bias and it is a sequential process

Ensemble Learning

- **Bagging** and **Boosting** are two types of **Ensemble Learning**. These two decrease the variance of a single estimate as they combine several estimates from different models. So the result may be **a model with higher stability**.
- **Bagging**: It is a **homogeneous weak learners' model** that learns from each other **independently** in parallel and combines them for determining the model **average**.
- **Boosting**: It is also **a homogeneous weak learners' model** but works differently from Bagging. In this model, learners learn **sequentially** and adaptively to **improve** model predictions of a **learning** algorithm.

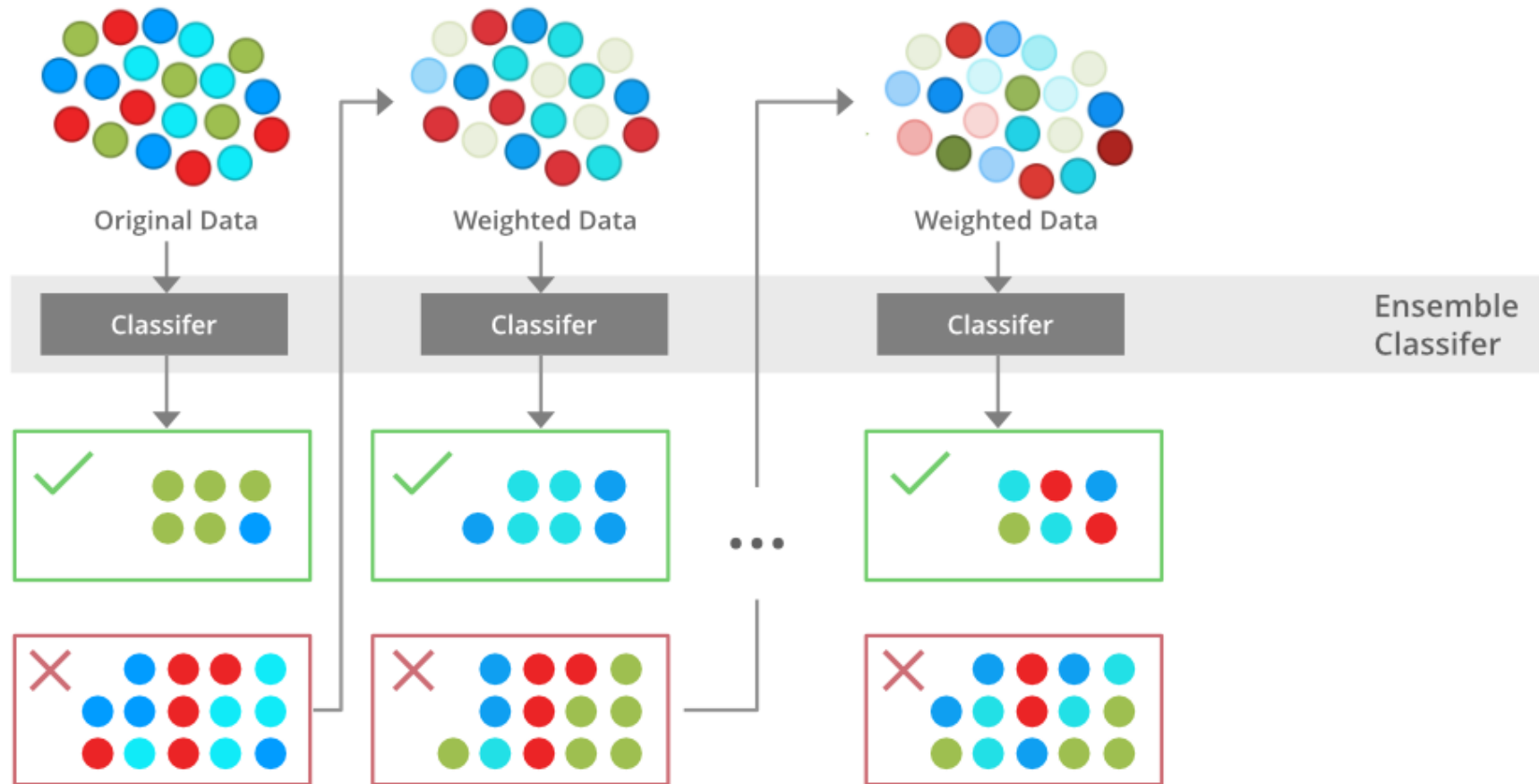
Bagging in Machine Learning



Bagging in Machine Learning

- **Bootstrap Aggregating**, also known as **bagging**, is a machine learning ensemble meta-algorithm designed to improve the stability and accuracy of machine learning algorithms used in **statistical classification and regression**.
- It **decreases the variance** and helps to **avoid overfitting**. It is usually applied to **decision tree methods**.
- Bagging is a special case of the model **averaging approach**.

Boosting in Machine Learning

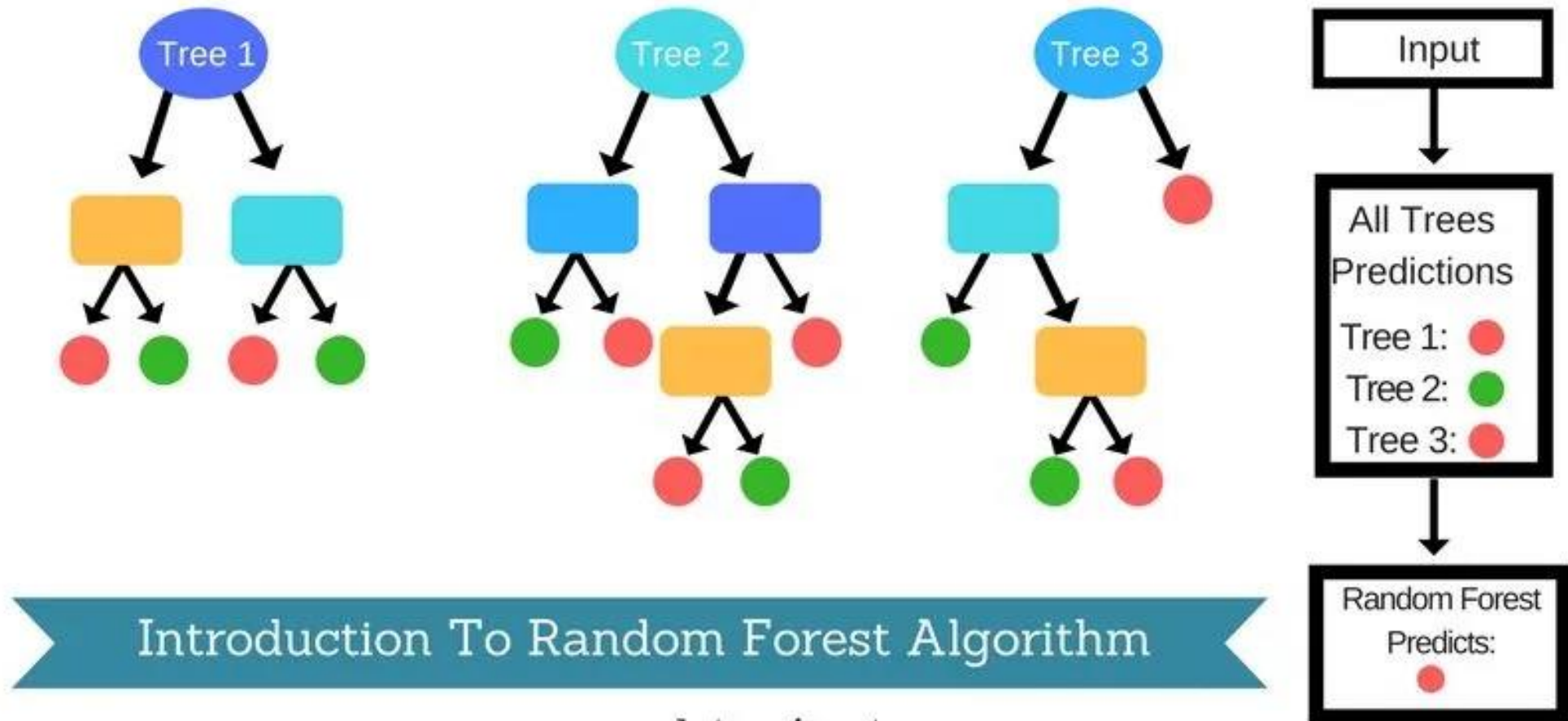


Boosting

- Boosting is the **ensemble learning method** where we build multiple weak learners (same algorithms) in a **SEQUENTIAL manner**.
- All these **weak learners** take the **previous models' feedback** to improve their **power** in accurately predicting the **missclassified classes**.
- Boosting algorithms are one of the **best-performing algorithms** among all the other Machine Learning algorithms with the **best performance** and **higher accuracies**.
- All the boosting algorithms work on the basis of **learning from the errors** of the **previous model trained** and **tried avoiding the same mistakes** made by the previously trained weak learning algorithm.

Random Forest

Random Forest



Introduction To Random Forest Algorithm

Random Forest

- The random forest algorithm is a **supervised classification** algorithm.
- As the name suggests, this algorithm creates the **forest** with a **number of trees**.
- In general, the **more trees in the forest** the more **robust** the forest looks like.
- In the same way in the random forest classifier, the **higher the number** of trees in the forest gives **the high the accuracy** results.
- In comparison, the random forest algorithm **randomly selects observations and features** to build **several decision trees** and then **averages** the results.

Why Random forest algorithm

- The same **random forest algorithm** or the random forest classifier can use for both **classification** and the **regression** task.
- Random forest classifier will **handle the missing** values.
- When we have more trees in the forest, a random forest classifier won't **overfit** the model.
- The random forest algorithm can be used for **feature engineering**.
 - This means identifying the **most important features** out of the available features from the training dataset.

Training data

Class	A	B	C
1	a1	b1	c1
2	a2	b2	c2
2	a3	b3	c3
1	a4	b4	c4
2	a5	b5	c5

Bootstrap

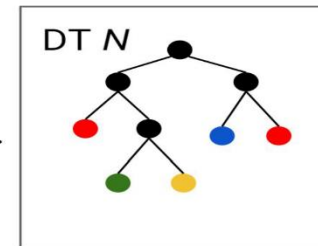
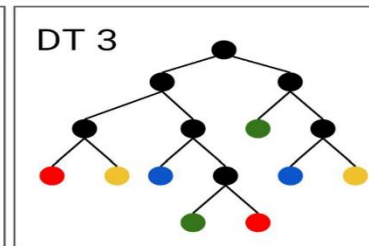
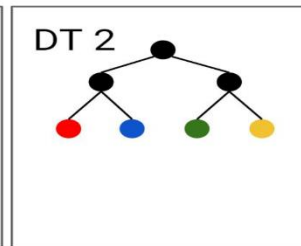
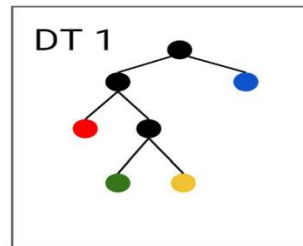
Class	A	B	C
1	a1	b1	c1
2	a2	b2	c2
2	a3	b3	c3
1	a4	b4	c4
1	a4	b4	c4

Class	A	B	C
2	a2	b2	c2
2	a2	b2	c2
2	a3	b3	c3
2	a5	b5	c5
1	a1	b1	c1

Class	A	B	C
1	a4	b4	c4
2	a5	b5	c5
1	a1	b1	c1
2	a5	b5	c5
2	a3	b3	c3

Bagging

Ensemble
of trees



Class 1

Class 2

Class 1

Class 1

Majority vote: Class 1



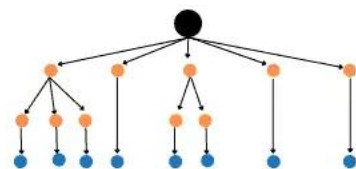
Dataset



Randomly Selected
Data 1



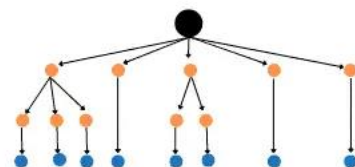
Decision tree 1



Randomly Selected
Data 2



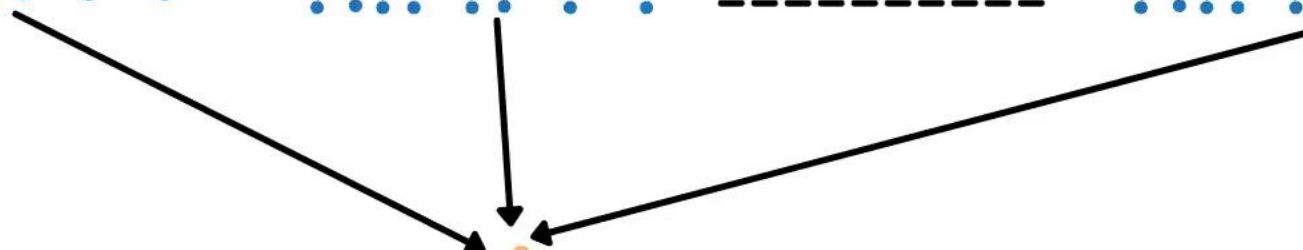
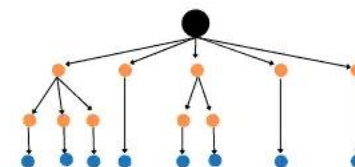
Decision tree 2



Randomly Selected
Data n



Decision tree n



Voting

Prediction



Training dataset

X_1	X_2	X_3	X_4	Y
a1	b1	c1	d1	1
a2	b2	c2	d2	2
a3	b3	c3	d3	1
a4	b4	c4	d4	1
a5	b5	c5	d5	2

Bootstrap

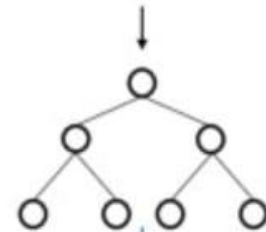
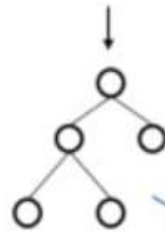
X_1	X_3	X_4	Y
a1	c1	d1	1
a2	c2	d2	2
a5	c5	d5	2

X_2	X_3	X_4	Y
b1	c1	d1	1
b3	c3	d3	1
b4	c4	d4	1

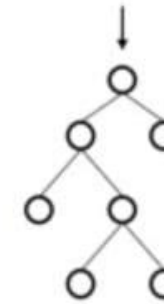
...

X_1	X_2	Y
a2	b2	2
a3	b3	1
a5	b5	2

Ensemble
of trees



...



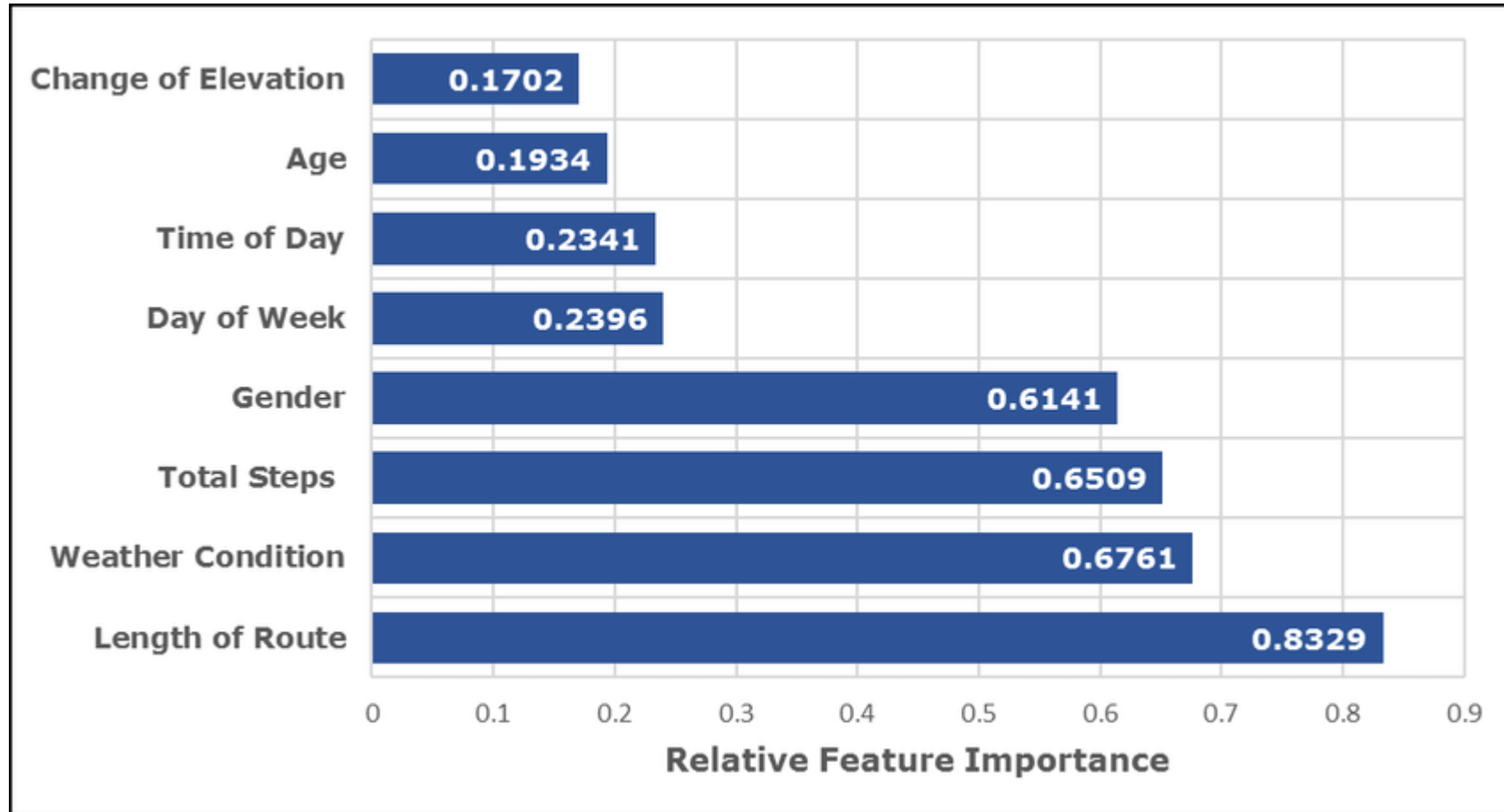
Aggregation

Majority decision

Random Forest

1. Takes the **test features** and use the rules of each randomly created decision tree to predict the outcome and stores the predicted outcome (target)
2. Calculate the **votes** for each predicted target.
3. Consider the **high voted** predicted target as the **final prediction** from the random forest algorithm.

Feature Importance



RANDOM FOREST DISADVANTAGES

- Increased accuracy requires **more trees**
- More trees **slow down** model

RANDOM FOREST IN Sklearn

- Firstly, there is the **n_estimators** hyperparameter, which is just the **number** of **trees** the algorithm builds before taking the maximum voting or taking the averages of predictions
- Another important hyperparameter is **max_features**, which is the **maximum** number of **features** random forest considers **to split** a **node**.
- The last important hyperparameter is **min_sample_leaf**. This determines the **minimum** number of **leafs** required to **split** an internal **node**.

RANDOM FOREST EVALUATION

Out-of-Bag (OOB) Error

How a Random Forest grades its own homework — no separate validation set required



STEP 1

Remember How a Random Forest Trains

Every tree in the forest trains on its own random draw from the data.

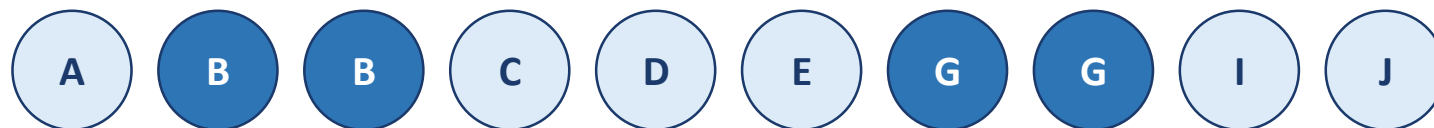
- **Bootstrap sampling:** rows are drawn with replacement
- Some rows get picked more than once
- Some rows are never picked at all

Those never-picked rows are the **Out-of-Bag (OOB)** samples for that tree.

Original Training Data (10 samples)



Tree 1's Bootstrap Draw (with replacement)



Darker circles = sampled more than once

Left Out of Tree 1 → Out-of-Bag



STEP 2

Tree 1 Has Never Seen F or H

Trained on

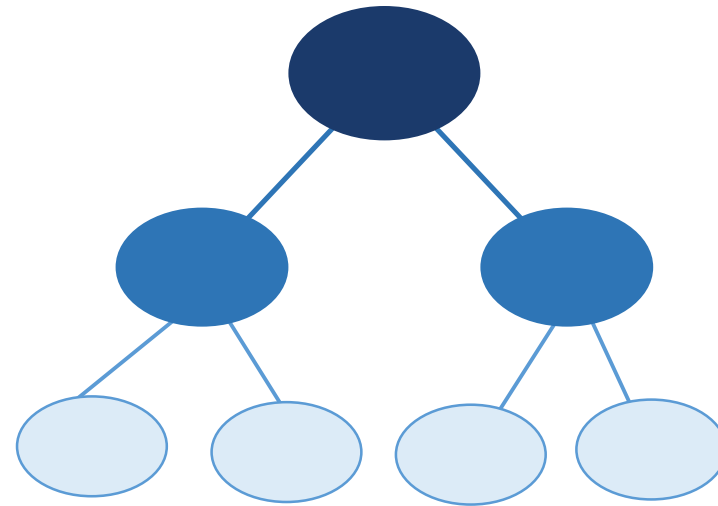
A B B C D E G G I J

Out-of-Bag (never trained on)

F H

Because Tree 1 never trained on F and H, it can be honestly tested on them

Tree 1



F and H sit outside every branch

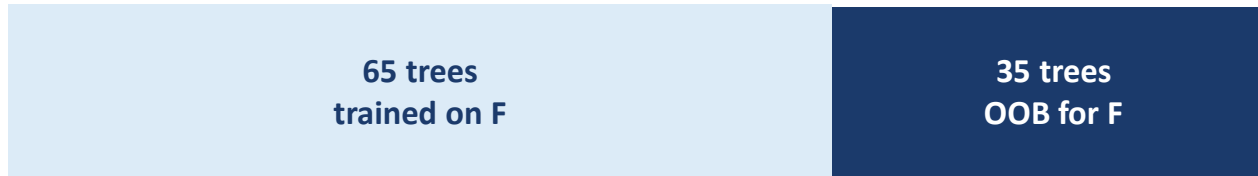
this tree learned from — they never influenced a single split.

STEP 3

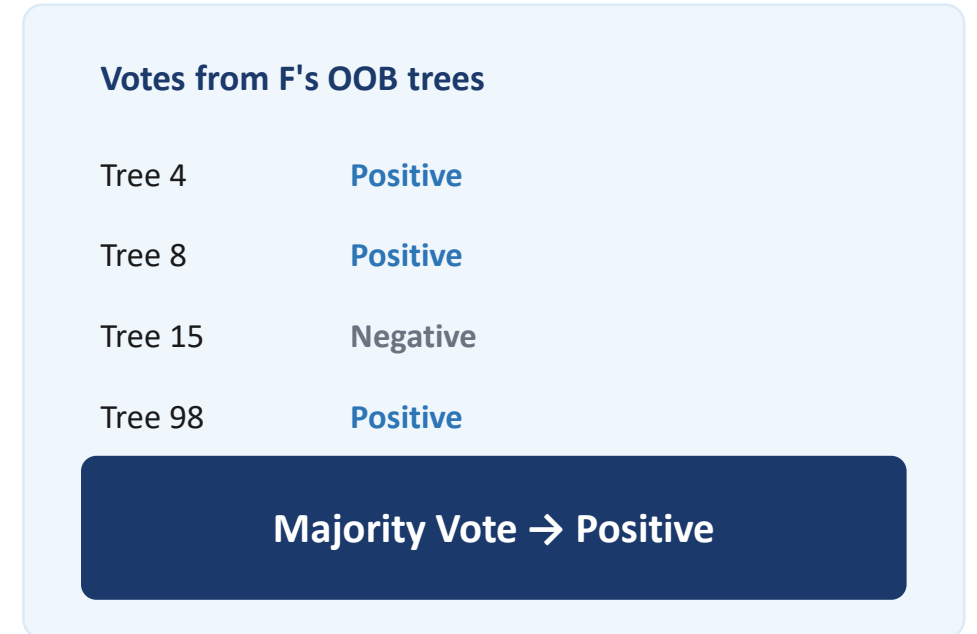
Scale This Up to the Whole Forest

Across 100 trees, each sample is trained-on by most trees and left out of the rest.

Sample F across 100 trees



Only those 35 trees are allowed to vote on F's prediction.



STEP 4

Calculating the OOB Error

Repeat the OOB vote for every training sample, then compare each result to its true label.

$$\text{OOB Error} = \frac{\text{incorrect OOB predictions}}{\text{total training samples}}$$

1000

training samples

930

predicted correctly

70 / 1000

0.07

OOB Error

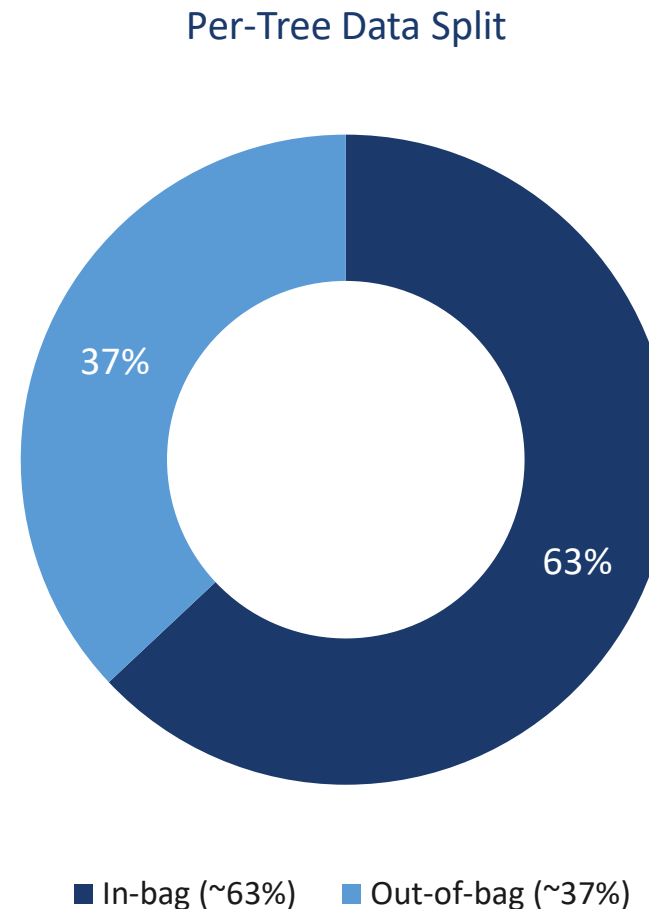
93% OOB Accuracy

Roughly 37% of the Data Is Left Out Per Tree

Bootstrap sampling with replacement means each tree sees only about **63%** of the training rows.

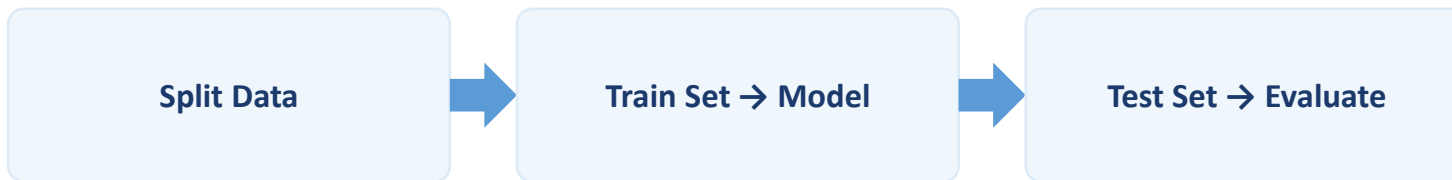
The remaining **37%** — the samples that tree never trained on — behave like unseen test data for it.

Averaged across the whole forest, that gives a reliable, built-in estimate of generalization performance.



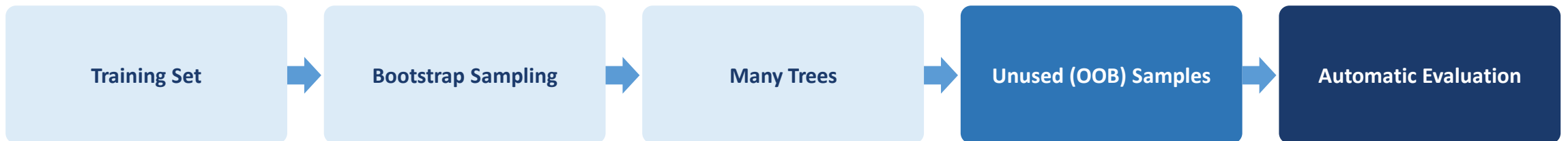
OOB Replaces a Separate Validation Split

Traditional Workflow



Needs data set aside purely for evaluation.

Random Forest with OOB



Performance is estimated automatically as a byproduct of training — no held-out data required.

Think of a Classroom, Not a Computer

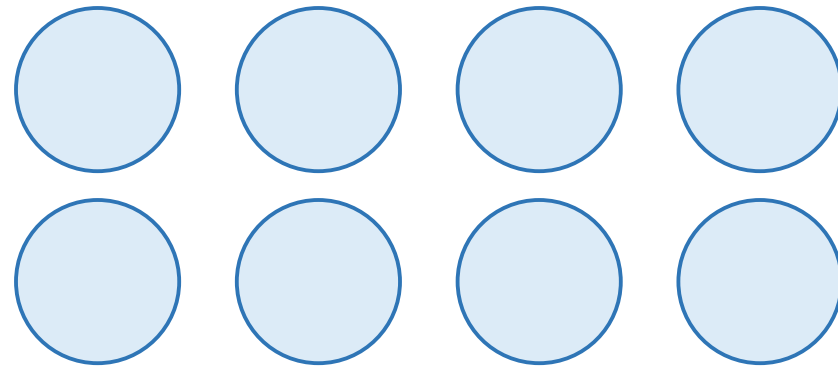
For every quiz, a teacher randomly picks some students to practice the exact questions in advance.

The rest of the class never sees those questions.

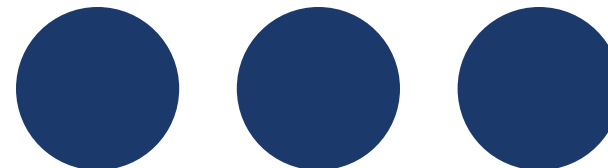
When it's time to check understanding, the teacher asks only the students who *didn't* practice.

Because they never rehearsed those questions, their scores give an honest read on real understanding — exactly what OOB error does for a tree.

Practiced the questions



Held out — asked to answer cold



These 3 give the honest score — just like a tree's OOB samples.

OOB Error vs. Test Error

OOB Error

- Uses the training data only
- Computed automatically while training
- Very good estimate of generalization
- Saves data — no split needed

Test Error

- Uses a separate, held-out test set
- Requires splitting the dataset first
- Usually treated as the final evaluation
- Reduces data available for training

Advantages & Limitations



Advantages

- No validation split needed — all data trains the model
- Computed automatically as a byproduct of training
- Close to k-fold cross-validation accuracy, at a fraction of the cost
- Handy for quick hyperparameter checks (e.g. number of trees)



Limitations

- Only applies to bagging-based ensembles — not boosting
- Needs enough trees; too few gives a noisy estimate
- Correlated trees can slightly bias the estimate
- Still worth a true held-out test set for a final, unbiased check

IN PRACTICE

```
from sklearn.ensemble import (  
    RandomForestClassifier  
)  
  
model = RandomForestClassifier(  
    n_estimators=300,  
    oob_score=True,  
    random_state=42  
)  
  
model.fit(X_train, y_train)  
  
print(model.oob_score_)  
# -> 0.93
```

Setting **`oob_score=True`** tells scikit-learn to track out-of-bag predictions while it fits every tree.

After training, **`model.oob_score_`** holds the OOB accuracy — an estimate of test performance you get for free.

No extra train/test split, no extra fitting — the estimate falls out of training itself.